INFO2222 Final Revision Guide

Usability, Security, and Usable Security

Generated from the provided INFO2222 lecture, tutorial, project, and exam-revision materials

Semester 1, 2026

Contents

1 Course Overview

1.1 What INFO2222 Is About

INFO2222 combines two bodies of knowledge that often pull against each other in real systems: **human-computer interaction** and **security**. The usability side asks whether people can understand, learn, remember, and complete tasks with a system. The security side asks whether the system protects confidentiality, integrity, availability, identity, and sensitive operations under adversarial conditions.

Key idea

The unit is not only about building interfaces or applying cryptographic mechanisms. It is about designing systems where security mechanisms can be correctly understood and used by ordinary users in realistic contexts.

1.2 Usability, Security, and Usable Security

Usability concerns effectiveness, efficiency, safety, utility, learnability, memorability, and user experience. **Security** concerns adversaries, assets, threats, vulnerabilities, risk, and controls. **Usable security** studies the interaction between the two: a security feature that users misunderstand, ignore, bypass, or cannot recover from may fail even if the underlying mechanism is technically sound.

Exam angle

The exam-revision lecture emphasises concepts, relevance, advantages and disadvantages, method selection, and application to scenarios. Expect questions that test whether you can choose a suitable method or mitigation and justify it.

1.3 Likely Exam Style

From the exam-revision material:

- The exam is a closed-book BYOD Canvas quiz with one permitted A4 double-sided note sheet.
- The writing time is 80 minutes plus 10 minutes reading time.
- There are 100 marks, with 40 multiple-answer questions: 30 questions worth 2 points and 10 questions worth 4 points.
- Negative marking means unsupported guessing is dangerous, especially on 4-point questions.
- Recall may be enough for a pass; higher marks require reflection, application, and method choice.

1.4 How To Use This Guide

Use Sections 2–4 to understand examinable concepts. Use Section 5 to see how tutorials turn concepts into reasoning patterns. Use Section 6 to map lectures to tutorial applications. Use Section 7 to reuse project work as exam evidence. Use Section 8 immediately before the exam for templates, tables, and MAQ strategy.

1.5 Exam Priority Tags

Major concepts are labelled with priority tags:

- **[HIGH]** core concepts likely to be examined directly or through scenarios.
- **[MEDIUM]** useful supporting concepts that help justify choices.

- **[LOW]** background/detail to recognise but not over-drill.

Following the W12 exam advice, formulas, exact numbers, angles, frequencies, and statistical calculations are not prioritised. Know what concepts such as Fitts' law, sensory limits, and t-tests are for, but do not spend revision time memorising equations or exact physiological values.

2 Usability and HCI Foundations

2.1 [HIGH] Usability and User Experience

Definition. Usability is the degree to which target users can use a system to achieve goals effectively, efficiently, safely, with sufficient utility, and with learnability and memorability. UX is broader: it includes subjective, emotional, aesthetic, and experiential qualities such as satisfaction, engagement, and trust.

Why it matters. A technically capable system can still fail if users cannot find functions, understand feedback, recover from mistakes, or trust the interface. In a security context, poor usability can create security failures: users reuse passwords, ignore warnings, select insecure defaults, or bypass confusing protections.

Tutorial/project application. W02 asks students to think about intuitive solutions and usability goals. The project applies this through requirements gathering, prototype design, and evaluation. A strong project answer links features to concrete user goals rather than saying only that an interface is "easy".

Exam angle

Typical exam transformation: given a system or interface description, identify which usability goal is weak, explain the consequence, and choose an evaluation or design response.

Common mistake

Do not treat usability as only "looks nice". Usability is about goal achievement under real task conditions. A visually polished login screen may still be unusable if errors are vague, MFA recovery is unclear, or password rules are not explained.

2.2 [HIGH] Usability Goals

Goal	Meaning	Security-related use	Common trap
Effectiveness	Users can complete the intended task accurately.	Users can actually complete login, consent, recovery, or secure message sending.	Calling a feature effective because it exists, without checking task success.
Efficiency	Users complete tasks with acceptable time and effort.	Security friction is not excessive; MFA and warnings do not block routine work unnecessarily.	Assuming fewer steps is always better; some security steps are justified.

Goal	Meaning	Security-related use	Common trap
Safety	Users are protected from serious errors and can recover.	Prevent accidental disclosure, wrong-recipient sending, weak settings, or irreversible security mistakes.	Treating safety only as physical safety.
Utility	The system provides the functions users need.	Security features cover real threats: authentication, encrypted transmission, certificate validation, access control.	Adding polished features that do not address user or security needs.
Learnability	New users can learn the system.	Users can understand password setup, MFA, permissions, warnings, and privacy choices.	Expecting users to read long manuals.
Memorability	Returning users can remember how to use it.	Users can remember secure workflows without writing passwords insecurely or bypassing controls.	Confusing memorability with memorising secrets.

2.3 [HIGH] Human-Computer Interaction

Definition. HCI studies the design, use, and evaluation of interactive systems for human users. It considers users, tasks, contexts, cognition, interfaces, and evaluation evidence.

Why it matters. INFO2222 uses HCI because secure systems are deployed in human environments. The interface is often where security succeeds or fails: warnings, authentication flows, permissions, and privacy settings all depend on user interpretation.

Answer template

For an HCI/security scenario: identify the user group and task; state the usability goal at risk; identify the security consequence; propose a design or evaluation method; justify the trade-off.

2.4 [HIGH] User-Centered Design

Definition. User-centered design is an iterative process that involves users, understands their goals and context, designs alternatives, prototypes, evaluates, and refines.

Why it matters. It reduces the risk of designing from developer assumptions. In usable security, UCD helps identify where users misunderstand security terms, ignore warnings, or cannot complete secure workflows.

Tutorial/project application. W03 and W04 support the project process: interviews, thematic analysis, personas, scenarios, and ideation. The project requires interview planning, themes, feature justification, low-fidelity prototypes, high-fidelity prototypes, and evaluation.

Common mistake

A user requirement is not the same as a feature wish. A requirement should be grounded in user needs, tasks, constraints, and context. “Add encryption” is a security requirement; “show users who can read this message before sending” is a usable-security design response.

2.5 [HIGH] Requirements, Users, Goals, Tasks, and Context

Definition. Requirements describe what the system should support. User goals are what people want to achieve. Tasks are actions performed to reach goals. Context includes environment, device, time pressure, collaboration setting, accessibility needs, and security risk.

Why it matters. Requirements guide design and evaluation. For security, context determines realistic threats and usability burdens: a student group-work system needs authentication, confidentiality, integrity, and recovery, but also fast coordination and clear collaboration visibility.

Exam angle

Exam questions may give a scenario and ask what data to collect, which users to recruit, or how to convert interview findings into features. Strong answers connect evidence to requirements and avoid unsupported design jumps.

2.6 [HIGH] Interviews and Thematic Analysis

Definition. Interviews gather qualitative data about user experience, needs, practices, and problems. Semi-structured interviews combine planned questions with follow-up flexibility. Thematic analysis identifies recurring patterns in qualitative data.

Why it matters. Interviews reveal why users behave as they do. In security, they can expose insecure workarounds, confusion about warnings, trust assumptions, and recovery problems that logs alone cannot explain.

Tutorial/project application. W03 trains interview planning and thematic analysis. The project requires a semi-structured interview plan, ethics consideration, recruitment, transcripts, themes, and feature justification. The interview material in the folder shows group-work issues such as visibility, coordination, duplicated work, inconsistent writing style, and deadline pressure; in exam use these should be treated as general design findings, not personal transcript detail.

Answer template

Interview-to-requirement answer: "The observed theme is *X*. It matters because users need to achieve *goal Y* in *context Z*. Therefore the system should support *feature or constraint A*. This can be evaluated by *method B* using *task C*."

2.7 [HIGH] Personas and Scenarios

Definition. A persona is a realistic representation of a user type, including goals, behaviours, needs, frustrations, and context. A scenario is a narrative of how a user uses a system to achieve a goal.

Why it matters. Personas and scenarios make requirements concrete. For security, they help reveal whether ordinary users will understand secure workflows: for example, who receives a recovery link, who can read a group message, or when a warning interrupts collaboration.

Tutorial application. W04 trains personas, scenarios, and requirements interpretation. A strong exam answer uses them to justify design decisions, not as decorative storytelling.

Common mistake

Do not create a persona with demographics only. A useful persona for INFO2222 includes goals, tasks, context, pain points, and security/privacy expectations.

2.8 [MEDIUM] Design Alternatives, Conceptual Design, and Concrete Design

Definition. Conceptual design defines what users can do and how the system is conceptually organised. Concrete design defines interface details: screens, widgets, layout, labels, and interaction sequences.

Why it matters. Conceptual design errors are hard to fix with visual polish. In usable security, conceptual decisions such as "who controls group access" or "where encryption status is shown" shape whether users form correct mental models.

Tutorial application. W04 uses the 10 plus 10 method to generate alternatives and then select or merge designs. Exam questions may ask which design alternative better supports a user goal or security property.

2.9 [HIGH] Prototyping

Definition. A prototype is an artefact used to explore, communicate, or evaluate design ideas. Low-fidelity prototypes are quick and incomplete; high-fidelity prototypes are more realistic in appearance and interaction.

Why it matters. Prototypes allow early testing before costly implementation. For security, prototypes can test whether users notice permission status, understand warnings, complete MFA, or recognise secure communication indicators.

Dimension	Low-fidelity prototype	High-fidelity prototype
Purpose	Explore ideas, layout, task flow, and conceptual alternatives.	Evaluate realistic interaction, visual hierarchy, and detailed behaviour.
Cost	Fast and cheap to change.	More expensive and can create attachment to details.
Best for	Early design, paper testing, Wizard-of-Oz, comparing alternatives.	Later usability testing, stakeholder demos, realistic interaction feedback.
Security use	Test whether users understand concepts such as permissions or recovery before implementation.	Test exact warning wording, login flow, MFA friction, certificate/error states.
Exam trap	Saying low-fi is inferior because it is rough.	Assuming hi-fi automatically means usable or secure.

2.10 [HIGH] Evaluation

Definition. Evaluation collects evidence about whether a system, prototype, method, or design meets goals. It may involve users or not, controlled settings or natural settings, qualitative or quantitative data.

Why it matters. Evaluation turns design claims into evidence. In security, evaluation can reveal whether users follow secure workflows, misunderstand warnings, or choose unsafe options under time pressure.

2.10.1 [HIGH] Usability Testing

Usability testing observes representative users performing concrete tasks. Data may include success, time, errors, comments, satisfaction, and observed confusion.

Security implication. Test tasks can include secure login, setting permissions, sending confidential messages, responding to certificate warnings, or recovering an account.

When to use. Use usability testing when a prototype or system exists and the question is whether target users can complete realistic tasks. It is especially useful for testing end-to-end workflows such as joining a group, setting permissions, logging in with MFA, or sending a private message.

When not to use. Do not use it as the first method when there is no task or prototype to test. Do not use it when the question is purely expert compliance, such as whether a screen follows a known checklist, unless user evidence is also needed.

Advantages. It gives direct evidence of task success, errors, hesitation, misunderstanding, and recovery behaviour. It can reveal security problems that technical inspection misses, such as users approving an unsafe prompt because the wording sounds routine.

Disadvantages. It needs participants, task design, time, and ethical care. Small tests find important issues but do not prove that every user will succeed.

Exam clue words. Representative users, concrete task, task success, errors, time-on-task, observation, prototype, user confusion.

Common MAQ traps. A usability test is not the same as a survey. It is not automatically quantitative. It does not replace security testing of the code. It does not prove universal usability from one or two participants.

Mini scenario. A group-chat prototype claims that file sharing is private. A usability test asks students to send a file to one group member and then explain who can see it. If several users choose the wrong recipient or misread the privacy label, the issue is both usability and confidentiality-related.

2.10.2 [HIGH] Think-Aloud Protocol

Think-aloud asks participants to verbalise thoughts while performing tasks. It reveals mental models, confusion, expectations, and decision points.

Tutorial/project application. W05 trains think-aloud process, task design, pitfalls, and reporting. The project requires a think-aloud study plan and findings table for successful and unsuccessful tasks.

Common mistake

Do not over-help participants during think-aloud. The goal is to discover what they understand unaided. Prompt them to keep talking, but do not teach them the interface.

When to use. Use think-aloud when the exam scenario asks how to understand users' mental models, expectations, decision points, or confusion during a task.

When not to use. Do not choose it when verbalisation would strongly distort the task, when the goal is a precise performance comparison, or when the participant cannot reasonably speak while doing the task.

Advantages. It exposes why users make choices, not just what they clicked. It is powerful for security UI because users may say, for example, "this warning looks like a normal update message" or "I think everyone in the group can read this".

Disadvantages. Speaking can slow users down, some participants give little verbal detail, and the evaluator must avoid leading prompts.

Exam clue words. Mental model, confusion, why users chose, verbalise, thoughts, expectations, task walkthrough.

Common MAQ traps. Think-aloud is not a tutorial session. The facilitator should not explain the interface, correct mistakes immediately, or ask leading questions like "Do you think this warning is dangerous?"

Mini scenario. A user sees a certificate warning and clicks through. Think-aloud reveals they believed "advanced" meant "recommended for advanced users", not "dangerous". The fix is warning wording and safer default flow, not only more cryptography.

2.10.3 [HIGH] Controlled Experiments

Controlled experiments compare conditions by manipulating independent variables and measuring dependent variables while controlling confounds. The lectures cover hypothesis, variables, participants, within/between-subjects designs, counterbalancing, and validity.

Security implication. A controlled experiment could compare two warning designs by measuring correct risk decisions, time, confidence, and false acceptance of risky actions.

When to use. Use a controlled experiment when the exam asks which design is faster, safer, more accurate, or more effective and there is a clear independent variable and dependent variable.

When not to use. Do not choose it for open-ended discovery of user needs, early ideation, or rich contextual understanding.

Advantages. It can support a stronger claim that one design caused better performance, especially when variables, participants, and order effects are controlled.

Disadvantages. It is artificial, more complex to design, and may miss real-world context. The W12 advice says not to memorise statistical calculations; know what tests are for, not how to calculate t-tests by hand.

Exam clue words. Compare, hypothesis, independent variable, dependent variable, control variable, within-subjects, between-subjects, counterbalancing.

Common MAQ traps. Do not call the measured outcome an independent variable. Do not assume correlation proves causation. Do not over-prioritise t-test formula calculation.

Mini scenario. To compare two MFA prompts, the independent variable is prompt design and dependent variables could be correct approval/rejection rate and completion time. Counterbalancing matters if the same participants see both prompts.

2.10.4 [MEDIUM] Field Studies

Field studies observe real-world use in natural context. Analytics/models can reveal behavioural patterns, performance, or predicted interaction cost. The lectures discuss models including Fitts' law at a conceptual level; the exam revision indicates key aspects matter more than memorising equations.

Security implication. Field evidence may show that users dismiss warnings during deadlines; analytics may show repeated failed logins or abandoned MFA setup.

When to use. Use a field study when context is central: real deadlines, group dynamics, device switching, noisy environments, or actual security workarounds.

When not to use. Do not choose it when the exam asks for tight control of a variable or a quick expert review.

Advantages. It has ecological validity and can reveal behaviour that does not appear in a lab.

Disadvantages. It is harder to control, compare, and analyse; ethics and privacy can be more complex.

Exam clue words. Natural setting, real use, context, workplace/classroom, long-term behaviour, diary, observation.

Common MAQ traps. Field study does not mean "more accurate in every way". It gives context but weaker control.

Mini scenario. Students ignore file-access warnings during a deadline because they are rushing. A field study can reveal deadline pressure as the cause; a lab test may only show that the warning text is readable.

2.10.5 [HIGH] Heuristic Evaluation

Heuristic evaluation is an expert inspection method that checks an interface against usability principles.

Security implication. Experts can inspect whether security warnings are visible, permission states are clear, error messages are actionable, and safe defaults are present.

When to use. Use heuristic evaluation for quick early feedback when user recruitment is difficult or before spending time on user testing.

When not to use. Do not use it as the only evidence when the question asks whether real users understand or can complete tasks.

Advantages. It is fast, relatively cheap, and can catch obvious violations before user testing.

Disadvantages. Findings depend on evaluator expertise; experts may miss issues that non-experts experience.

Exam clue words. Expert review, principles, inspection, no users, quick evaluation, usability heuristics.

Common MAQ traps. Heuristic evaluation is not the same as usability testing. It does not directly measure user success or satisfaction.

Mini scenario. An expert notices that the "share with all" button is visually more prominent than "share with selected members". This is a safety/default issue that could cause accidental disclosure.

2.10.6 [HIGH] Cognitive Walkthrough

A cognitive walkthrough inspects whether a user can discover and complete the correct action sequence, especially on first use.

When to use. Use it when the exam scenario focuses on learnability, discoverability, first-time use, or whether users know what to do next.

When not to use. Do not choose it for measuring speed across many users or for broad emotional UX evaluation.

Advantages. It focuses tightly on task steps and user knowledge at each step.

Disadvantages. It depends on assumptions about user knowledge and may not reveal real emotional reactions or social context.

Exam clue words. First-time user, discover, next step, learnability, walkthrough, action sequence.

Common MAQ traps. It is not just "walking through the app". The evaluator asks whether the user would know the goal, find the action, connect the action to the goal, and interpret feedback.

Mini scenario. A first-time user needs to revoke another member's access. A walkthrough asks whether they would know to open group settings, find member permissions, choose revoke, and understand the confirmation message.

2.10.7 [MEDIUM] Inspection

Inspection is a broader expert checking approach. It may include heuristic evaluation, walkthroughs, checklists, consistency checks, accessibility checks, or security-specific UI review.

When to use. Use inspection when the exam asks for expert review without users, checklist-based review, or early detection of obvious interface risks.

When not to use. Do not use it when the key uncertainty is actual user comprehension or behaviour.

Advantages. It is efficient and can cover many screens or states quickly.

Disadvantages. It can produce false confidence if treated as a substitute for user evidence.

Exam clue words. Checklist, expert, review, inspect, consistency, compliance.

Common MAQ traps. Inspection can find likely problems, but it does not show how representative users behave under real task pressure.

Mini scenario. Inspectors review all error messages and find that certificate failures say only "connection failed", which hides the security meaning from users.

2.10.8 [MEDIUM] Analytics and Models

Analytics use logs or behavioural data; models predict or explain aspects of interaction. In this unit, model details such as Fitts' law should be understood conceptually, not as equation memorisation.

When to use. Use analytics/models when the scenario mentions logs, repeated behaviour, abandonment, frequency, time patterns, or predicted interaction cost.

When not to use. Do not use them alone when you need to understand users' reasons, mental models, or qualitative experience.

Advantages. They scale across many users and can reveal patterns invisible in small tests.

Disadvantages. Logs show what happened, not necessarily why. Models simplify human behaviour.

Exam clue words. Logs, metrics, frequency, clickstream, repeated failed attempts, model, predicted time.

Common MAQ traps. Do not memorise Fitts' law equation or exact sensory values. Know the implication: target size and distance affect pointing difficulty.

Mini scenario. Analytics show 60 percent of users abandon MFA setup at the backup-code screen. This identifies a problem location; think-aloud or interviews may explain the misunderstanding.

3 Security Foundations

3.1 [HIGH] Security Modelling

Definition. Security modelling identifies assets, adversaries, goals, capabilities, assumptions, defender goals, and controls. A system is only as strong as its weakest relevant part.

Why it matters. Security answers are meaningless unless they specify what is being protected from whom. A control that stops passive eavesdropping may not stop a malicious server or phishing attack.

Usability implication. If the model ignores user behaviour, it may choose controls that users bypass. For example, a strong password policy may encourage unsafe password storage if it is too hard to use.

Answer template

Threat model: asset; attacker; attacker capability; attacker goal; assumptions; vulnerability; likely impact; control; residual usability cost.

3.2 [HIGH] CIA Triad

Property	Meaning	INFO2222-style example	Common trap
Confidentiality	Prevent unauthorised disclosure.	Encrypt group messages; prevent database inference of salary; protect passwords in transit.	Confusing secrecy with correctness.
Integrity	Prevent unauthorised modification or detect tampering.	MAC/signature on messages; SQL injection prevention; certificate validation.	Assuming encryption alone proves integrity.
Availability	Ensure authorised users can access service when needed.	Defend against DoS/SYN flood; rate limiting; resilient login.	Ignoring availability because it is not about secrecy.

Usability implication. Strong confidentiality may add friction; integrity checks may create warnings; availability controls such as rate limiting must avoid locking out legitimate users unfairly.

3.3 [HIGH] Hashing, Salting, and Password Storage

Definition. A cryptographic hash maps input to a fixed-size digest. Good password storage stores salted slow password hashes, not plaintext passwords. A salt is a per-user random value that prevents identical passwords from having identical stored hashes and makes precomputed tables less useful.

Problem solved. If the password database is stolen, attackers should not immediately learn plaintext passwords.

What it does not solve. Hashing does not encrypt data for later decryption. It does not protect passwords sent over plaintext networks. It does not stop phishing, weak user choices, or credential stuffing by itself.

Assumptions. The hash function and slow password-hashing configuration remain computationally expensive for attackers; salts are unique and stored correctly; users still choose or manage secrets.

Usability implication. Password rules and slow hashing affect registration/login experience. Excessively strict policies can lead to reuse or insecure notes; good design gives clear requirements, password-manager support, and recovery paths.

Violated property if absent. Plaintext or weak password storage primarily threatens confidentiality of user credentials, and then accountability and integrity because stolen credentials allow impersonation and unauthorised actions.

Mitigation. Use a password-hashing scheme with a unique salt per password and a cost parameter that slows offline guessing. Transmit passwords only over a secure channel, and combine with rate limiting and MFA for online attacks.

Common MAQ trap. Salts do not need to be secret; they need to be unique and random enough to defeat shared precomputation. Hashing a password on the client only is not enough if the transmitted hash becomes a password-equivalent secret.

Mini scenario. An attacker steals a user table. If it contains plaintext passwords, confidentiality is immediately lost. If it contains fast unsalted MD5 hashes, common passwords can be cracked quickly. If it contains salted slow hashes, the attacker must pay a separate high offline cost per account.

Common mistake

Do not say "encrypt passwords in the database" when the requirement is password verification. Passwords should normally be salted and hashed with a password-hashing scheme; the server verifies by hashing the submitted password and comparing.

3.4 [MEDIUM] Rainbow Tables

Definition. Rainbow tables precompute hash/reduction chains to trade storage for faster password cracking. The W9 tutorial uses rainbow tables to explain why naive unsalted hashing is weak.

Exam relevance. You should understand the concept and why salting defeats shared precomputation. The implementation exercise is marked not examinable, so treat code mechanics as supporting background only.

Usability implication. Users with common passwords are at higher risk; systems should support password managers and avoid policies that push users toward predictable patterns.

Problem solved by attacker. Rainbow tables reduce online cracking work by precomputing chains for many possible passwords.

What salts change. A unique salt makes the same password produce different stored hashes for different users, so one shared rainbow table no longer applies to all accounts.

Assumptions. Rainbow tables are useful only when the password space and hash setup are known and precomputation is reusable.

Mitigation. Use unique salts and slow password hashing. Do not rely on users choosing rare passwords unaided.

Common MAQ trap. A rainbow table is not magic decryption; it is a time-storage trade-off for guessing likely preimages.

Mini scenario. If two users both choose "password123" and the site stores unsalted hashes, their hashes match. With salts, the stored values differ and precomputed endpoints cannot be reused directly.

3.5 [HIGH] Symmetric Encryption and One-Time Pad

Definition. Symmetric encryption uses the same secret key for encryption and decryption. The one-time pad provides perfect secrecy when the key is truly random, as long as the message, and never reused.

Problem solved. It protects message confidentiality from an eavesdropper who lacks the key.

What it does not solve. It does not solve key distribution, authentication, integrity, or endpoint compromise by itself.

Assumptions. Keys remain secret, random/secure enough, and correctly managed. For OTP, key reuse breaks security.

Usability implication. Key management is hard for users. Perfect secrecy is impractical for ordinary systems because distributing and storing one-time keys is burdensome.

Violated property if absent. Without encryption, confidentiality of data in transit or at rest can be violated by eavesdroppers or unauthorised readers.

Mitigation. Use modern symmetric encryption through vetted libraries and protocols. For large messages, use secure modes or authenticated encryption rather than inventing a mode.

Common MAQ trap. Symmetric encryption is efficient, but it assumes both parties already share a secret key. OTP is perfectly secret only under strict conditions; reusing an OTP key leaks relationships

between messages.

Mini scenario. Alice and Bob use the same OTP key for two messages. An attacker who XORs the two ciphertexts removes the key and learns information about the two plaintexts. The failure is key reuse, not the XOR operation itself.

3.6 [HIGH] Asymmetric Encryption and Public-Key Cryptography

Definition. Asymmetric cryptography uses a public/private key pair. Public-key encryption lets anyone encrypt to a recipient's public key, while only the private-key holder can decrypt.

Problem solved. It reduces the need to share a secret key before communication and supports encryption, digital signatures, and secure-channel setup.

What it does not solve. It does not automatically prove that a public key belongs to the intended person. Without authentication, a man-in-the-middle can substitute keys.

Assumptions. Private keys remain private; public keys are authenticated; mathematical problems remain hard enough.

Usability implication. Users rarely manage raw public keys well. Interfaces need understandable identity verification, certificate status, and recovery mechanisms.

Violated property if absent. Without authenticated public keys, confidentiality and integrity can fail through man-in-the-middle substitution.

Mitigation. Use certificates, verified key directories, fingerprints, or trusted channels to bind public keys to identities.

Common MAQ trap. Public-key encryption does not by itself prove the public key belongs to Bob. It only ensures that whoever owns the matching private key can decrypt.

Mini scenario. Alice downloads a public key labelled "Bob" from an untrusted page. Mallory replaced it with her own key. Alice's message is encrypted, but to Mallory. The missing property is authenticated key ownership.

3.7 [HIGH] Diffie-Hellman Key Exchange

Definition. Diffie-Hellman allows parties to derive a shared secret over an insecure channel. Ephemeral variants support forward secrecy.

Problem solved. It establishes shared keys without sending the key itself.

What it does not solve. Plain DH does not authenticate parties; it can be vulnerable to man-in-the-middle attacks.

Usability implication. Users need assurance about who they are communicating with, but should not have to understand modular arithmetic. Good systems hide key exchange while exposing meaningful identity/security state.

Assumptions. The chosen group and parameters are secure, private exponents remain private, and the exchanged public values are authenticated by some external mechanism if active attackers are in scope.

Violated property if attacked. A man-in-the-middle can break confidentiality and integrity by establishing separate keys with each party.

Mitigation. Authenticate the exchange using certificates, signatures, pre-shared keys, or verified public keys. Ephemeral DH can add forward secrecy, but authentication is still needed.

Common MAQ trap. DH hides the shared secret from passive eavesdroppers, but unauthenticated DH does not stop an active attacker from sitting in the middle.

Mini scenario. Alice and Bob exchange DH values in a chat app. Mallory substitutes her own values in both directions. Alice shares a key with Mallory, Bob shares a different key with Mallory, and both believe they share a key with each other.

3.8 [HIGH] MACs, HMAC, Digital Signatures, and Authenticated Encryption

Definition. A MAC uses a shared secret key to verify message authenticity and integrity. A digital signature uses a private signing key and public verification key, enabling public verifiability and non-repudiation-like evidence. Authenticated encryption combines confidentiality and integrity.

Problem solved. These mechanisms detect tampering and authenticate message origin under the relevant key model.

What they do not solve. A MAC does not identify which party produced a message if both share the same key. A signature does not hide content. Authenticated encryption does not fix wrong-recipient selection or endpoint compromise.

Usability implication. Users should not be asked to manually decide cryptographic composition. Interfaces should communicate message authenticity in task language, such as "verified sender" or "message may have been altered".

Assumptions. MAC keys and signing keys must remain secret. Verification keys must be correctly bound to identities. Authenticated encryption must use nonces/parameters correctly through a safe library.

Violated property if absent. Without MACs, signatures, or authenticated encryption, integrity and authenticity can fail even if confidentiality appears protected.

Mitigation. Use HMAC or an authenticated-encryption scheme where appropriate. Use digital signatures when public verifiability or identity binding is needed.

Common MAQ trap. Encryption alone is not a message-authentication guarantee. A MAC is not the same as a digital signature because the verifier also has the MAC key.

Mini scenario. A server receives an encrypted command "transfer 10". If encryption is unauthenticated, an attacker may tamper with ciphertext and cause a meaningful change. Authenticated encryption makes tampering detectable before processing.

MAC/HMAC vs signatures. Use HMAC when parties share a secret and only those parties need to verify messages. Use a digital signature when anyone with the public key should verify that the private-key holder signed the message. In MAQs, look for clue words such as "shared secret", "public verification", "non-repudiation-like evidence", or "anyone can verify".

Authenticated encryption. Authenticated encryption protects confidentiality and integrity together. The exam trap is to treat "encrypt-then-ignore-integrity" as equivalent. Correct reasoning says the receiver should reject modified ciphertext before using plaintext.

3.9 [HIGH] Identification, Authentication, Authorization, and Accountability

Identification asks who the user claims to be. **Authentication** verifies the claim. **Authorization** decides what the verified user may do. **Accountability** supports tracing actions to responsible identities.

Problem solved. These concepts structure access control: know the claimed identity, verify it, restrict actions, and audit behaviour.

What they do not solve. Authentication does not imply permission. Authorization does not prove identity. Accountability does not prevent all harm; it supports deterrence and investigation.

Usability implication. Login, MFA, roles, permissions, and audit notifications must be understand-

able. Poor permission design can cause over-sharing or blocked collaboration.

Assumptions. Identity evidence is reliable, credentials or tokens are protected, permission rules match policy, and logs are trustworthy.

Violated property if absent. Weak authentication enables impersonation; weak authorization enables unauthorised access; weak accountability weakens investigation and deterrence.

Mitigation. Separate identity verification from permission checks. Use role-based or least-privilege access where appropriate, and keep audit logs for important actions.

Common MAQ trap. "The user is logged in" answers authentication, not authorization. A logged-in student may still not be authorised to read another group's private messages.

Mini scenario. A student logs into the group-work system successfully. That proves authentication. Whether they can delete another student's file is an authorization question.

3.10 [HIGH] Password, Token, Biometric, and Multi-Factor Authentication

Definition. Authentication factors include something you know, have, or are. MFA combines factors to reduce dependence on one secret.

Problem solved. MFA reduces risk from stolen or guessed passwords.

What it does not solve. MFA may not stop phishing or social engineering if users approve malicious prompts. Biometrics cannot be changed like passwords and may raise privacy/accessibility issues.

Usability implication. MFA can improve security but add friction, failure modes, recovery complexity, and accessibility concerns. Good design uses clear prompts, secure defaults, fallback paths, and avoids warning fatigue.

Assumptions. The factors are independent enough that compromise of one does not automatically compromise the others. Recovery channels are protected.

Violated property if absent. Weak authentication threatens confidentiality, integrity, and accountability because attackers can act as legitimate users.

Mitigation. Use MFA for sensitive accounts, rate limit online guessing, use secure session tokens, and design recovery carefully.

Common MAQ trap. MFA is not automatically secure against all phishing. If users are trained to approve push prompts blindly, an attacker can exploit prompt fatigue.

Mini scenario. A student receives repeated MFA prompts during an attack and approves one to stop the notifications. The technical control exists, but the usable-security failure allows account compromise.

3.11 [HIGH] TLS, HTTPS, Certificates, and E2EE

Definition. TLS secures a transport connection, authenticating the server through certificates and deriving session keys for encrypted communication. HTTPS is HTTP over TLS. End-to-end encryption protects content between endpoints so intermediaries cannot read it.

Problem solved. TLS protects against passive network eavesdropping and many active network attacks when certificate validation is correct. E2EE protects message content from servers or intermediaries.

What it does not solve. TLS does not protect against a malicious endpoint or server that legitimately sees plaintext. E2EE does not automatically authenticate user identity or protect metadata.

Assumptions. Certificate authorities or pinned keys are trusted; clients validate certificates; endpoints

are not compromised.

Usability implication. Certificate errors, padlocks, and encryption indicators are easily misunderstood. Secure defaults matter because ordinary users should not be responsible for validating low-level cryptographic details.

TLS/HTTPS problem solved. TLS protects HTTP traffic against passive eavesdropping and many active network attackers by authenticating the server and deriving session keys.

TLS/HTTPS does not solve. It does not protect data from the server itself, from malicious JavaScript served by the site, from endpoint malware, or from users sending data to the wrong recipient.

Certificate assumptions. The certificate chain, host name, validity period, and trust anchor must validate. Certificate pinning or hardcoded CA keys can reduce some risks but introduce update and recovery problems.

Violated property if broken. Failed certificate validation can violate confidentiality and integrity through MITM. Availability may also suffer if overly strict certificate handling blocks legitimate recovery after certificate rotation.

Mitigation. Use TLS 1.2+ or newer correctly, validate certificates, avoid disabling validation, and present certificate errors as rare, serious, and actionable.

Common MAQ trap. HTTPS is not the same as E2EE. With normal HTTPS, the server terminates TLS and can read plaintext.

Mini scenario. A login page sends passwords over HTTPS, but the app accepts any self-signed certificate without warning. An attacker on public Wi-Fi can impersonate the server, so the secure-channel claim fails.

E2EE problem solved. E2EE keeps message content confidential from servers and intermediaries by encrypting at the sender endpoint and decrypting at the recipient endpoint.

E2EE does not solve. It does not hide all metadata, fix weak endpoint security, prove the recipient's key identity automatically, or stop users from forwarding plaintext.

E2EE usability implication. Users need simple indicators for recipient identity and device changes. If key verification is too hard, users may ignore it; if warnings are too frequent, they may dismiss real key-substitution attacks.

3.12 [MEDIUM] Network Attacks: DoS, DDoS, Amplification, SYN Flood

Definition. Denial-of-service attacks reduce availability. SYN flood exploits TCP connection setup by creating many half-open connections. Amplification attacks use third-party systems to multiply traffic.

Problem solved by controls. Rate limiting, infrastructure scaling, filtering, timeouts, and anti-amplification controls reduce availability risk.

What they do not solve. Availability controls do not protect confidentiality or integrity. Overly strict controls can block legitimate users.

Usability implication. Defences must distinguish legitimate high demand from attacks; otherwise users experience unfair lockouts or degraded service.

Assumptions. The defender can detect abnormal traffic or resource consumption and has enough infrastructure or filtering support to respond.

Violated property. DoS and SYN flood attacks mainly violate availability.

Mitigation. Use rate limiting, SYN cookies or similar TCP protections, timeouts, filtering, load balancing, and upstream DDoS mitigation.

Common MAQ trap. DoS is not primarily about stealing data. It can still be a serious security issue because availability is part of the CIA triad.

Mini scenario. A server receives many SYN packets but the handshakes are not completed. Its connection table fills with half-open connections, preventing legitimate users from connecting.

Heartbleed distinction. Heartbleed is not a DoS attack. It is a memory disclosure bug caused by trusting a supplied heartbeat length, allowing attackers to read process memory.

Heartbleed mitigation. Patch the vulnerable library, validate bounds, rotate potentially exposed keys, and invalidate affected credentials or sessions where needed.

Heartbleed MAQ trap. The violated property is confidentiality. The attacker may recover private keys, tokens, passwords in memory, or session fragments if they are nearby in process memory.

3.13 [HIGH] Database Security, Inference Attacks, and SQL Injection

Inference attacks combine allowed aggregate query results to infer sensitive individual information. Small result rejection reduces risk but may fail if overlapping queries are allowed. Mitigations include query auditing, noise/partial disclosure, views, and restricting query granularity.

SQL injection occurs when untrusted input is treated as executable SQL. Mitigations include parameterised queries, validation, sanitisation, least privilege, and avoiding string concatenation.

Usability implication. Privacy-preserving policies can frustrate legitimate analysts if too restrictive. SQL injection defences should be built into developer workflows so secure coding is the easy default.

Inference attack problem solved by policy. Small-result rejection tries to stop users from directly querying tiny groups and learning individual facts.

What it does not solve. If the system allows repeated overlapping aggregate queries, an attacker may subtract results and infer a target's value.

Inference assumptions. The attacker may know some background facts, such as department membership, and can issue multiple accepted aggregate queries.

Violated property. Inference attacks violate confidentiality and privacy even though the system never directly reveals raw rows.

Inference mitigation. Limit overlapping queries, use views at safe granularity, add noise or partial disclosure, audit query patterns, and restrict attributes available to users.

Inference MAQ trap. "The query returns at least six rows" is not sufficient if multiple accepted queries can be combined.

Inference mini scenario. A salary database rejects queries with fewer than six people. Fred queries all IT salaries plus one known outsider, then queries the same group without Adam using another accepted aggregate. Subtracting results can reveal Adam's salary.

SQL injection problem solved by controls. Parameterised queries keep user data separate from SQL structure.

What SQL injection controls do not solve. Parameterisation does not automatically fix weak authorization, poor password storage, or XSS.

SQL injection assumptions. The attack needs an input path that reaches a query unsafely and a database account with enough privilege to cause harm.

Violated property. SQL injection can violate confidentiality by reading data, integrity by modifying data, and availability by deleting or locking data.

SQL injection mitigation. Use prepared statements, validation, least-privilege database accounts, careful error handling, and secure code review.

SQL injection MAQ trap. Escaping or sanitising strings manually is weaker as a general answer

than parameterised queries.

SQL injection mini scenario. A login form builds SQL by concatenating the username. Input like a quote plus SQL logic changes the query meaning. The correct fix is not a stronger password rule; it is safe query construction.

3.14 [HIGH] XSS and Web Security

Definition. Cross-site scripting allows attackers to inject script into pages viewed by users. The W11 tutorial uses the Google XSS game to build intuition about unsafe input handling.

Problem solved by controls. Output encoding, sanitisation, content security policy, safe frameworks, and careful handling of user-controlled content reduce XSS risk.

What it does not solve. XSS controls do not prevent SQL injection, weak authentication, or network eavesdropping.

Usability implication. Rich text, previews, and user-generated content are usability features that increase attack surface. Secure design must preserve useful expression while preventing script execution.

Assumptions. The browser will execute script in the page context unless output is encoded or blocked. Users may trust content because it appears inside a legitimate site.

Violated property. XSS can violate confidentiality by stealing tokens or data, integrity by performing actions as the user, and user trust by altering displayed content.

Mitigation. Encode output by context, sanitise allowed HTML, use safe templating, avoid dangerous DOM APIs, apply Content Security Policy, and protect cookies where appropriate.

Common MAQ trap. XSS is not the same as SQL injection. XSS executes in the user's browser; SQL injection changes database queries.

Mini scenario. A comment field stores text that is later rendered as HTML. An attacker posts a script that runs when classmates view the page. The failure is output handling, not password hashing.

CSRF status

The provided PDF text did not show clear CSRF coverage. Treat CSRF as related background rather than a confirmed examinable focus unless your instructor separately identifies it.

3.15 [MEDIUM] Program Analysis

Definition. Static analysis inspects code without running it. Dynamic analysis observes execution. Symbolic analysis reasons about program paths using symbolic inputs.

Problem solved. These methods find potential bugs, vulnerabilities, or risky paths.

What they do not solve. Static analysis may report false positives; dynamic analysis depends on tested executions; symbolic analysis may not scale.

Usability implication. Developer-facing security tools need usable output. If warnings are noisy, unclear, or hard to prioritise, developers may ignore them.

Assumptions. Static analysis assumes the code representation and rules approximate real vulnerabilities. Dynamic analysis assumes tested inputs and executions are representative. Symbolic analysis assumes the explored model is accurate enough.

Violated property found. Program analysis can reveal vulnerabilities that would later violate confidentiality, integrity, or availability, depending on the bug.

Mitigation role. These methods are detection and assurance tools. They help find bugs; they are

not themselves runtime security controls.

Common MAQ trap. Static analysis does not run the program. Dynamic analysis does run it but may miss untested paths. Symbolic analysis reasons about paths but can face path explosion.

Mini scenario. Static analysis flags unsanitised data flowing into an HTML sink. Dynamic testing confirms an XSS payload executes. A useful developer tool should explain the source, sink, and fix in a way developers can act on.

4 Usable Security

4.1 [HIGH] Core Trade-Off

Security mechanisms impose requirements on users: remember secrets, verify identities, respond to warnings, grant permissions, recover accounts, and make privacy decisions. Usability failures can therefore become security failures.

Key idea

Human error should often be treated as a design problem. A secure system should make the safe action visible, understandable, and practical.

4.2 [HIGH] Security Warnings and Warning Fatigue

Warnings are useful only when users understand the risk, believe the warning is relevant, and have a clear safe action. Repeated vague warnings create habituation and dismissal.

Answer template

Warning analysis: What risk is the warning about? What decision must the user make? Is the language specific? Is the safe option clear? What happens if the user is busy, stressed, or non-expert?

4.3 [HIGH] Passwords and MFA

Strict password policies may increase theoretical strength while worsening real behaviour through reuse, predictable substitutions, or insecure storage. MFA improves security against password theft but adds device dependency, recovery burden, prompt fatigue, and accessibility issues.

4.4 [HIGH] Privacy Versus Convenience

Convenient defaults can disclose data. Privacy-preserving controls can reduce utility. Strong answers identify both sides and propose balanced mitigations such as secure defaults, progressive disclosure, clear consent, and role-based access.

4.5 [HIGH] Secure Defaults, Least Privilege, and User Control

Secure defaults make the safe option automatic. **Least privilege** gives users/systems only the access needed. **User control** lets users make meaningful choices, but excessive choice can overwhelm.

Common mistake

Do not answer "give users full control" as if it always improves usability. Too many security choices can increase cognitive load and misconfiguration.

4.6 [HIGH] Case-Study Answer Pattern

Answer template

Given a system:

1. Identify the usability issue in task terms.
2. Identify the security risk or failed security property.
3. Explain the trade-off.
4. Propose a mitigation.
5. Discuss the user impact and residual risk.

5 Tutorial-Based Revision

5.1 W02: Usability Goals and Project Framing

What skill this tutorial trains. Turning a broad problem into an intuitive, usable solution and applying usability goals.

Lecture concepts used. Usability, UX, design goals, HCI, user-centered design, project framing.

Typical exam transformation. Given a proposed interface, identify which usability goals it supports or violates and justify a design improvement.

Key reasoning pattern. Start with the user goal; identify task barriers; map each barrier to effectiveness, efficiency, safety, utility, learnability, or memorability; propose a change.

Common mistake. Listing features without explaining which user need or usability goal they support.

Key exercise types. Brainstorming usable solutions, identifying intuitive features, explaining usability goals, and connecting project ideas to users.

Expected reasoning steps. Identify target users; state their goal; describe the task barrier; choose the relevant usability goal; propose a design feature; justify why it helps.

MAQ or scenario form. An MAQ may ask which usability goal is most directly affected by an interface problem. A scenario question may ask how to improve a confusing group-work feature.

Wrong-answer patterns. Treating aesthetics as usability; selecting every usability goal without prioritising; proposing features without user evidence.

Mini exam-style example. A file-sharing button is labelled "publish" and students are unsure whether it sends to one person or the whole group. The strongest answer identifies a safety/confidentiality risk and proposes clearer recipient visibility before sending.

5.2 W03: Interviews and Thematic Analysis

What skill this tutorial trains. Gathering qualitative data and converting it into themes and requirements.

Lecture concepts used. Requirements, data gathering, interviews, data recording, thematic analysis.

Typical exam transformation. Choose an appropriate data-gathering method, write or critique an interview plan, or infer requirements from user statements.

Key reasoning pattern. Evidence quote or observation → code → theme → requirement → design implication.

Common mistake. Treating one participant comment as a universal requirement without triangulation or justification.

Key exercise types. Creating interview plans, distinguishing structured/semi-structured/open interviews, recording data, coding responses, grouping codes into themes, and deriving requirements.

Expected reasoning steps. Define the study goal; select participants and ethics process; prepare open but focused questions; collect data; code repeated ideas; form themes; turn themes into justified requirements.

MAQ or scenario form. An MAQ may ask which data-gathering method fits early requirements work. A scenario may give interview snippets and ask which theme or requirement is best supported.

Wrong-answer patterns. Asking leading questions; confusing a theme with a feature; inventing requirements not supported by the data; ignoring ethics and consent.

Mini exam-style example. Several participants say group members duplicate work because progress is invisible. A strong answer forms the theme "coordination visibility" and derives a requirement for progress/status awareness, not simply "add chat".

5.3 W04: Personas, Scenarios, and Ideation

What skill this tutorial trains. Bringing requirements to life through personas, scenarios, and alternative designs.

Lecture concepts used. Personas, scenarios, conceptual design, concrete design, alternatives, 10 plus 10 ideation, prototyping.

Typical exam transformation. Select between design alternatives or explain how a persona/scenario supports design reasoning.

Key reasoning pattern. Persona goal/context → scenario task → design requirement → prototype feature → evaluation criterion.

Common mistake. Creating personas that are demographic profiles rather than task-focused design tools.

Key exercise types. Building personas from requirements, writing user scenarios, using 10 plus 10 ideation, comparing alternatives, and deciding between conceptual/concrete designs.

Expected reasoning steps. Start from user evidence; define persona goals and pain points; write a scenario around a task; generate alternatives; compare how each alternative supports goals, usability, and security.

MAQ or scenario form. An MAQ may ask what a persona is used for. A scenario may ask which design alternative better supports a user goal or reduces a security mistake.

Wrong-answer patterns. Listing demographics only; treating ideation as the final answer; selecting a design because it looks modern rather than because it fits requirements.

Mini exam-style example. A persona often works near deadlines and misses permission details. A suitable scenario should include time pressure and access decisions so the design can be tested for safe defaults.

5.4 W05: Think-Aloud Evaluation

What skill this tutorial trains. Designing and reporting usability evaluation using think-aloud.

Lecture concepts used. Evaluation, usability testing, think-aloud, concrete tasks, qualitative feedback, reporting, ethics.

Typical exam transformation. Choose think-aloud for a scenario, design suitable tasks, identify

pitfalls, or interpret findings.

Key reasoning pattern. Define goal; recruit suitable participants; give realistic tasks; observe without teaching; record success, errors, comments, and breakdowns; convert findings into design changes.

Common mistake. Asking users whether they like the interface instead of observing whether they can complete meaningful tasks.

Key exercise types. Writing think-aloud tasks, choosing concrete tasks over abstract goals, prompting without leading, recording observations, and reporting successful/unsuccessful tasks.

Expected reasoning steps. Define the evaluation goal; choose participants; create realistic tasks; ask users to verbalise; observe errors and comments; separate observation from interpretation; recommend design changes.

MAQ or scenario form. An MAQ may test whether the facilitator should help. A scenario may ask how to evaluate whether users understand a security warning or permission state.

Wrong-answer patterns. Teaching during the session; using vague tasks like "explore the app"; reporting only opinions; ignoring failed tasks.

Mini exam-style example. Task: "Send a message that only your project partner can read." If users say "I think the whole group can see this" while selecting the wrong option, the finding is a mental-model and confidentiality issue.

5.5 W7: OTP, Hashing, and Key Reuse

What skill this tutorial trains. Reasoning about cryptographic properties, especially perfect secrecy, XOR, hashing, and key reuse.

Lecture concepts used. Security modelling, Kerckhoff's principle, randomness, hash functions, symmetric encryption, one-time pad, key reuse.

Typical exam transformation. Explain why OTP conditions matter, compare hashing and encryption, or identify why reusing keys is dangerous.

Key reasoning pattern. State the security property; state the assumption; show what breaks when the assumption fails; name the mitigation.

Common mistake. Saying "OTP is perfectly secure" without the conditions: truly random key, equal length, secret key, and one-time use.

Key exercise types. XOR encryption/decryption, explaining perfect secrecy, identifying key-reuse failure, applying Kerckhoff's principle, and distinguishing hashing from encryption.

Expected reasoning steps. State the cryptographic goal; state the required assumption; apply the mechanism; explain what breaks if the assumption is violated.

MAQ or scenario form. An MAQ may ask which OTP condition is required. A scenario may ask why two ciphertexts produced with the same key leak information.

Wrong-answer patterns. Treating hashing as reversible; saying secrecy depends on hiding the algorithm; ignoring randomness and key distribution.

Mini exam-style example. A system uses the same random-looking pad for every message. Select the option saying perfect secrecy no longer holds because the one-time-use assumption is violated.

5.6 W8: PKE, MACs, Signatures, and Key Exchange

What skill this tutorial trains. Distinguishing cryptographic mechanisms by purpose and key model.

Lecture concepts used. Public-key encryption, symmetric vs asymmetric encryption, MAC/HMAC, digital signatures, Diffie-Hellman, authenticated encryption.

Typical exam transformation. Choose the right mechanism for confidentiality, integrity, authentication, or public verifiability.

Key reasoning pattern. Identify the security goal; identify who holds which key; decide whether secrecy, shared-key authenticity, public verification, or key establishment is required.

Common mistake. Confusing encryption with authentication, or MACs with signatures.

Key exercise types. Public-key encryption demonstrations, symmetric/asymmetric comparison, HMAC reasoning, signature reasoning, Diffie-Hellman key exchange, and authenticated-encryption ordering.

Expected reasoning steps. Identify the goal: confidentiality, integrity, shared-key authenticity, public verifiability, or key establishment. Then choose the mechanism with the matching key model.

MAQ or scenario form. An MAQ may ask which mechanism provides public verification. A scenario may ask why unauthenticated key exchange permits MITM.

Wrong-answer patterns. Saying a public key is secret; saying a MAC proves authorship to outsiders; assuming PKE automatically authenticates the recipient's key.

Mini exam-style example. Alice wants anyone to verify that Bob approved a project submission. A digital signature fits better than an HMAC because public verification is needed.

5.7 W9: Rainbow Tables and TLS

What skill this tutorial trains. Understanding password-cracking trade-offs and secure-channel establishment.

Lecture concepts used. Password hashing, salting, rainbow tables, TLS handshake, certificates, session keys, HTTPS.

Typical exam transformation. Explain why salts defeat rainbow tables, why TLS needs certificate validation, or how TLS differs from E2EE.

Key reasoning pattern. For password storage: stolen database $\rightarrow$ offline attack $\rightarrow$ salt and slow hash reduce attack efficiency. For TLS: server certificate $\rightarrow$ key exchange $\rightarrow$ symmetric session protection.

Common mistake. Treating TLS as end-to-end encryption for messages when the server is still an endpoint for TLS.

Key exercise types. Rainbow-table concept, password-hash cracking logic, salts, TLS handshake roles, certificates, session keys, and HTTPS boundaries.

Expected reasoning steps. For passwords, identify offline attack conditions and how salts/slow hashes change attacker cost. For TLS, identify authentication, key exchange, and encrypted record protection.

MAQ or scenario form. An MAQ may ask why salts help. A scenario may ask what goes wrong when certificate validation is disabled.

Wrong-answer patterns. Saying salts encrypt passwords; treating certificate validation as optional; saying TLS means the server cannot read the message.

Mini exam-style example. A developer disables certificate checks to make testing easier and forgets to re-enable them. The resulting risk is MITM, violating confidentiality and integrity of the login channel.

5.8 W10: Inference Attacks, SQL Injection, and HTTPS

What skill this tutorial trains. Applying threat modelling to database and network scenarios.

Lecture concepts used. Inference attacks, partial disclosure, views, SQL injection, validation/sanitisation, HTTPS/TLS, TCP/IP layering.

Typical exam transformation. Given a database policy or login form, identify the attack, explain why it works, and propose mitigations.

Key reasoning pattern. Identify allowed interface; identify attacker goal and prior knowledge; show how outputs can be combined or input can be interpreted unsafely; add controls at the right layer.

Common mistake. Assuming that hiding individual rows is enough privacy protection when aggregate queries can be combined.

Key exercise types. Inference attack reasoning with accepted aggregate queries, partial disclosure, views, SQL injection diagnosis, and HTTPS/TCP/TLS layering.

Expected reasoning steps. Define attacker goal and prior knowledge; identify what the interface allows; combine outputs or unsafe inputs; state violated property; propose a mitigation.

MAQ or scenario form. An MAQ may ask why small-result rejection is insufficient. A scenario may give a vulnerable SQL query and ask for the best defence.

Wrong-answer patterns. Saying "use HTTPS" to fix SQL injection; saying aggregate data is always safe; using manual string filtering as the main SQLi defence.

Mini exam-style example. A system rejects queries over fewer than six people but allows unlimited overlapping sums. The correct answer identifies an inference risk and suggests limiting overlap or using safe views/noise.

5.9 W11: XSS, SYN Flood, Heartbleed, and Analysis Methods

What skill this tutorial trains. Recognising common vulnerability classes and matching them to causes and defences.

Lecture concepts used. XSS, TCP handshake, SYN flood, Heartbleed, memory disclosure, static analysis, dynamic analysis, symbolic analysis.

Typical exam transformation. Match an attack description to the correct vulnerability, explain the failed assumption, and propose a mitigation.

Key reasoning pattern. Attack symptom $\rightarrow$ root cause $\rightarrow$ violated property $\rightarrow$ mitigation $\rightarrow$ usability or developer-workflow impact.

Common mistake. Treating all web attacks as input validation only. XSS, SQL injection, Heartbleed, and SYN flood affect different layers and properties.

Key exercise types. XSS payload reasoning, TCP handshake and SYN flood recognition, Heartbleed memory disclosure, and comparing static, dynamic, and symbolic analysis.

Expected reasoning steps. Classify the attack; identify the layer and root cause; state the violated CIA property; choose a mitigation; mention usability or developer-workflow impact where relevant.

MAQ or scenario form. An MAQ may ask which description matches SYN flood or Heartbleed. A scenario may ask whether static or dynamic analysis is better for a given vulnerability-finding task.

Wrong-answer patterns. Calling Heartbleed SQL injection; saying SYN flood violates confidentiality first; assuming static analysis executes the program; treating XSS and SQLi as the same because both involve input.

Mini exam-style example. A web page stores comments and later renders them as active script. The correct classification is XSS, the likely property impact includes confidentiality/integrity, and the

mitigation is context-aware output encoding plus sanitisation.

6 Lecture-to-Tutorial Mapping

Lecture topic	Tutorial	Key concepts	Application type	Exam relevance
Intro usability and UX	W02	Usability goals, HCI, intuitive design	Identify goals and design issues	Scenario classification
UCD and requirements	W03	Interviews, data recording, thematic analysis	Convert user evidence into requirements	Method choice and justification
Design and prototyping	W04	Personas, scenarios, alternatives, lo-fi/hi-fi prototypes	Bring requirements into design	Choose/justify design method
Evaluation	W05	Think-aloud, tasks, metrics, reporting	Plan and interpret user evaluation	Evaluation design questions
Human factors and accessibility	W02–W05	Attention, perception, memory, accessibility, WCAG principles	Explain user limits and inclusive design	Concept application; no numeric memorisation
Usability and security	W05, project	Balance, warnings, alternative paths, vulnerability mitigation	Explain trade-offs	Usable-security scenarios
Crypto foundations	W7	OTP, XOR, randomness, key reuse, hashing	Mechanism assumptions	Concept boundaries
PKC and auth crypto	W8	PKE, DH, MAC, signatures, AE	Select crypto mechanism	Goal-to-mechanism mapping
Authentication and TLS	W9	Password storage, salts, TLS, certificates	Secure login and transport	Compare TLS/E2EE; password storage
Network and database security	W10	Inference attacks, SQL injection, HTTPS	Threat model and mitigation	Attack and defence scenarios
Software and system security	W11	XSS, Heartbleed, SYN flood, program analysis	Vulnerability recognition	Match cause, property, mitigation
Exam revision	All	MAQ strategy, method selection, concepts over details	Apply and justify	Negative marking strategy

7 Project / Assignment Concepts

7.1 Reusable UCD Concepts

The project requires a web-based communication system for student group work. Exam-useful concepts include:

- **Requirements gathering:** plan interviews, define participants, consider ethics, record data, analyse themes.
- **Feature justification:** connect features to themes and compare against existing tools.
- **Scenario writing:** explain how a target user completes a realistic task.
- **Prototyping:** use lo-fi alternatives before committing to hi-fi design.
- **Evaluation:** plan think-aloud tasks and report successful/unsuccessful tasks.

7.2 Reusable Security Concepts

The project requires authentication, message confidentiality, message integrity, certificate validation, secure password transmission, and E2EE. It also asks students to demonstrate vulnerabilities by disabling or misconfiguring controls.

Answer template

Security design justification: "The asset is A. The threat is T. The vulnerability without the control is V. The selected control C protects *property* P. Its usability impact is U, which can be reduced by *design* D."

7.3 Common Rubric Traps

- Stating a security feature without explaining the threat it addresses.
- Demonstrating a vulnerability without explaining why the mitigation fixes it.
- Treating LLM-generated interfaces as correct without evaluation.
- Writing broad claims rather than concise evidence-based reasoning.
- Ignoring usability effects of security controls.

8 Exam Toolkit

8.1 MAQ Exam Strategy and Negative Marking

Key idea

In a multiple-answer question, each option should be judged independently. Do not look for exactly one correct answer unless the question explicitly says so.

- Read the stem first and identify the concept being tested.
- For each option, ask: "Is this always true under the stated assumptions, or only sometimes true?"
- Avoid guessing. The exam material explicitly warns that negative marks make guessing harmful, especially for 4-point questions.
- Be conservative on 4-point questions: select an option only when you can justify it from a lecture/tutorial concept.
- Watch absolute wording: **always**, **never**, **cannot**, **only**, **must**. Such options may be correct for definitions or strict security assumptions, but are often traps when real-world exceptions exist.

- Separate mechanism from goal: encryption, hashing, authentication, authorization, TLS, and E2EE solve different problems.
- If two options sound similar, locate the boundary: user involvement vs no user, confidentiality vs integrity, server-authenticated channel vs endpoint-only secrecy.

Answer template

MAQ option test: "This option is correct only if *assumption*. The question states *given fact*. Therefore I should select / not select it because *concept boundary*."

8.2 Do Not Overlearn

The W12 exam advice explicitly says to focus on main concepts, relevance, advantages and disadvantages, method choice, and application. Do not spend final revision time over-learning details that the exam advice de-emphasises.

- **No sketching.** You should understand design and prototyping concepts, but the exam is not expected to ask you to draw interface sketches.
- **No direct brainstorming/design-thinking phase listing.** Know why ideation and alternatives matter; do not memorise brainstorming phases as a list.
- **No exact sensory numbers.** For human factors, know attention, perception, memory, recognition versus recall, accessibility, and the design implications. Do not memorise exact angles, frequencies, or mechanoreceptor ranges.
- **No statistical calculations.** Know what hypotheses, variables, validity, p-values, and t-tests are for. Do not practise hand-calculating a t-test unless your instructor separately changes the advice.
- **No Fitts' law equation memorisation.** Know the conceptual implication: smaller or farther targets are harder/slower to acquire; larger and closer targets are easier. Do not memorise the equation.

Exam angle

Spend time on: concept boundaries, method selection, advantages/disadvantages, UCD application, usability-security trade-offs, and scenario reasoning. In MAQs, the correct option often depends on whether the statement is true under the stated assumptions.

8.3 Method Selection Table

Method	Use when	Do not use when	Advantages	Cons	Clue words
Usability testing	Need evidence from representative users doing tasks.	Only expert judgement is needed or no prototype/task exists.	Direct task evidence; finds real breakdowns.	Recruiting cost; limited sample.	users, tasks, success, errors
Controlled experiment	Need compare conditions and infer effect of variable.	Exploratory early design or uncontrolled context matters more.	Stronger causal claims.	Artificial setting; design complexity.	compare, variable, hypothesis

Method	Use when	Do not use when	Advantages	Cons	Clue words
Field study	Need natural context and real behaviour.	Need tight control or quick lab feedback.	Ecological validity; reveals context.	Less control; harder analysis.	real setting, natural use
Heuristic evaluation	Need quick expert review against principles.	Need actual user mental models or task evidence.	Fast; no users required.	Depends on evaluator skill; may miss real-user issues.	expert, principles, inspection
Cognitive walkthrough	Need assess learnability of task steps.	Need measure performance or satisfaction.	Good for first-time-use problems.	Step-by-step; assumption-heavy.	learn, discover, first time
Inspection	Need systematic expert checking.	Need user evidence.	Efficient; broad coverage.	Not a substitute for user testing.	review, inspect, checklist
Analytics and models	Need logs or predicted interaction cost.	Need explain why users think or feel something.	Scales; quantitative patterns.	May miss motivation and context.	logs, model, frequency, time

8.4 Security Concept Boundary Table

Boundary	Left side	Right side	Exam trap
Hashing vs encryption	Hashing is one-way verification/digesting.	Encryption is reversible with key.	Saying hashed passwords can be decrypted.
TLS vs E2EE	TLS protects transport link to server.	E2EE protects content between endpoints.	Assuming HTTPS hides content from the server.
Authentication vs authorization	Authentication verifies identity.	Authorization grants permissions.	Login does not imply access to everything.
MAC vs digital signature	MAC uses shared secret; verifier can also create MAC.	Signature uses private/public key; public verification.	Claiming MAC gives public proof of origin.

Boundary	Left side	Right side	Exam trap
Symmetric vs asymmetric	Same secret key for encrypt/decrypt or MAC.	Public/private key pair.	Ignoring key distribution or identity verification.
Threat vs vulnerability vs risk	Threat is potential cause/attacker/event.	Vulnerability is weakness; risk combines likelihood and impact.	Calling every bad outcome a vulnerability.

8.5 Required Comparison Tables

8.5.1 Usability Testing vs Heuristic Evaluation

	Usability testing	Heuristic evaluation
Participants	Representative users	Experts/evaluators
Evidence	Observed task performance	Principle-based issues
Best for	Real user breakdowns	Quick early review
Security use	Test warning/login/permission comprehension	Inspect security UI clarity
Trap	Small test is not universal proof	Expert review is not user evidence

8.5.2 Authentication vs Authorization

	Authentication	Authorization
Question	Who are you? Can you prove it?	What are you allowed to do?
Example	Password/MFA/token login	Role permissions, group access
Failure	Impersonation	Privilege escalation/over-sharing
Usability issue	Login friction/recovery	Permission confusion

8.5.3 Hashing vs Encryption

	Hashing	Encryption
Direction	One-way	Reversible with key
Use	Password storage, integrity digest	Confidentiality of stored/transmitted data
Key	Usually no key; password hashing uses salt/cost	Requires key management
Trap	Not decryption	Encryption alone may not authenticate

8.5.4 Symmetric vs Asymmetric Encryption

	Symmetric	**Asymmetric**
Keys	Shared secret	Public/private pair
Strength	Efficient for bulk data	Solves some pre-shared-key problems
Weakness	Key distribution	Public-key authenticity and performance
Use	Session encryption	Key exchange, PKE, signatures

8.5.5 TLS vs End-to-End Encryption

	TLS/HTTPS	**E2EE**
Scope	Device to server channel	Sender endpoint to recipient endpoint
Server visibility	Server usually sees plaintext	Server should not see content
Depends on	Certificate validation and key exchange	Endpoint keys and identity verification
Trap	Not protection from server	Not automatic metadata protection

8.5.6 XSS vs SQL Injection

	XSS	**SQL injection**
Where attack executes	Browser/page context	Database query context
Cause	User input rendered as script	User input treated as SQL code
Impact	Session theft, actions as user, content manipulation	Data read/write/delete, auth bypass
Mitigation	Output encoding, sanitisation, CSP	Parameterised queries, validation, least privilege

8.6 Definition Bank

- **Effectiveness:** accuracy/completeness of task success.
- **Efficiency:** resources/time required for task success.
- **Learnability:** ease for new users to begin using a system.
- **Memorability:** ease for returning users to re-establish proficiency.
- **Threat:** potential cause of unwanted incident.
- **Vulnerability:** weakness that can be exploited.
- **Risk:** likelihood and impact of harm.
- **Salt:** per-password random value stored with a password hash.
- **Certificate validation:** checking certificate chain/name/trust before relying on TLS identity.

8.7 Scenario Answer Templates

Answer template

Usability evaluation: goal; participants; tasks; method; metrics/data; procedure; ethics; expected findings; design response.

Answer template

Security analysis: asset; attacker; threat; vulnerability; affected CIA property; mitigation; assumption; usability impact.

Answer template

Trade-off discussion: security benefit; user cost; likely user behaviour; mitigation to reduce burden; residual risk.

8.8 Final Checklist

- Can I explain each concept in one sentence and one scenario?
- Can I distinguish mechanisms with similar names?
- Can I choose an evaluation method from clue words?
- Can I state both security and usability implications?
- For MAQs, can I justify every selected option?